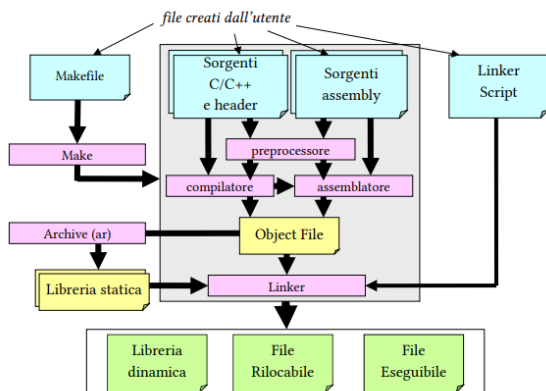


C e asm

venerdì 27 febbraio 2026 11:41

1. Installare compilatore su linux
 - a. `$ sudo apt-get install build-essential`
2. Installare debugger linux
 - i. `$ sudo apt-get install gdb`
3. Ulteriori strumenti :
 - a. Objdump (disassemblatore)
 - i. Dalla codifica binaria , produce il codice in assembler
 - b. Valgrind:
 - i. Macchina virtuale che mostra errori nell'uso della memoria
 - c. Strings
 - i. Mostra tutte le stringhe presenti nel codicestruc

Vediamo ora il processo di compilazione :



Il pre-processor fa la traduzione da codice sorgente a codice sorgente modificato (magari leva commenti ecc ecc), il quale dato in pasto al compilatore vero proprio e cerca di trasformare la sequenza di byte in hardware, il quale viene trasformato in altro file di testo (assembler), il quale viene prodotto l'oggetto in assembler e lo trasforma in file oggetto (sequenza di byte : non ancora eseguito). Infine si arriva al linker, il quale collega più file oggetto, formando così librerie dinamiche, file oggetto oppure eseguibile. Andiamo a vedere in dettaglio : partiamo da un semplicissimo file.c che stampa a schermo un messaggio :

1. `#include <stdio.h>` Chiediamo al pre-processor di importare un altro file
2. `/* This is a comment. */` Un commento multilinea
3. `int main(int argc, char *argv[])` I vostri programmi partono sempre dalla funzione main. Il sistema operativo carica il programma e lo lancia da qui.
4. `{` Inizio di un blocco
5. `int times = 100;` Per funzionare, deve restituire un int ed accettare due parametri: un int per il numero di parametri e un array di stringhe char * per i parametri
6. Dichiarazione e assegnazione di variabile locale
7. `// this is also a comment` Un commento a linea singola
8. `printf("Hello World! I welcome you %d times.\n", times);`
9. `return 0;` Chiamata a funzione. È una funzione strana, con un numero arbitrario di parametri
10. `}` Valore di ritorno della funzione. In questo caso è il valore di ritorno al sistema operativo
11. La fine di un blocco

La riga 1 chiede al pre-processor di includere la libreria nel nostro file. Due tipi di include : quello della riga 1 che va a cercare la libreria nelle cartelle di sistema, oppure quella tra virgolette : va a cercare la libreria anche nelle stesse cartelle. Da notare che i commenti possono essere multi-linea o singola linea. Per quanto riguarda il main è la funzione principale del nostro programma ed il sistema operativo gli delega l'inizio delle attività. Per quanto riguarda i parametri : il primo è un integer (contatore) che rappresenta in numero dei parametri, i quali parametri sono passati nel secondo parametro : vettore di stringhe. Il "%d" rappresenta il valore di una qualche variabile (nel nostro caso è intero). Per quanto riguarda il valore di ritorno : se 0 tutto ok, altrimenti se diverso vi è errore. Andiamo a vedere ora il debugger : strumento che si attacca ad altro programma e ne permette l'interruzione in modo selettivo : con questo strumento è possibile sia ispezionare lo stato del programma, ma anche di modificare i dati.

In dettaglio :

Compilate aggiungendo l'opzione -g

Per lanciare il debugger: `gdb --args ./program [args]`

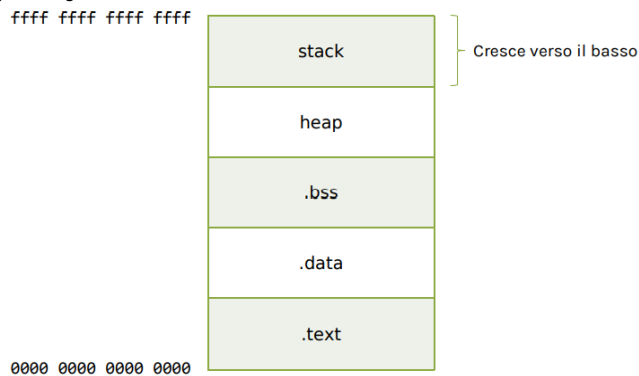
Comandi principali:

- `run [args]`: avvia il programma passando gli argomenti [args].
- `break [file:]function`: imposta un breakpoint.
- `backtrace`: mostra un backtrace delle funzioni chiamate fin'ora.
- `print: expr`: mostra il valore di expr.
- `continue`: continua l'esecuzione del programma.
- `next`: vai alla prossima riga nel sorgente, ma non entrare nelle funzioni.
- `step`: vai alla prossima riga nel sorgente, entrando nelle funzioni.
- `step instruction`: esegui una singola istruzione assembly
- `quit`: termina l'esecuzione del debugger.

Modalità interattiva: `CTRL-X, a (o --tui)`

- `layout regs`: mostra una finestra con il contenuto dei registri
- `layout asm`: mostra il sorgente assembly
- `layout src`: mostra il sorgente C
- `focus [what]`: sposta il focus su una particolare finestra

Per quanto riguarda invece al memoria :



la sezione .bss rappresenta tutte le variabili inizializzate a 0 . Nell'heap invece ci sono altre variabili . Ed in dettaglio :

```

1. #include <stdio.h>
2.
3. /* This is a comment. */
4. int main(int argc, char *argv[])
5. {
6.     int times = 100;
7.
8.     // this is also a comment
9.     printf("Hello World! I
        welcome you %d times.\n",
        times);
10.    return 0;
11. }
    
```

```

.data
.LC0:
.ascii "Hello World! I welcome
        you %d times.\n"

.text
main:
    pushq %rbp
    movq %rsp, %rbp
    subq $8, %rsp
    movl $100, -4(%rbp)
    movl -4(%rbp), %esi
    leaq .LC0(%rip), %rdi
    xorl %eax, %eax
    call printf@PLT
    xorl %eax, %eax
    addq $8, %rsp
    leave
    ret
    
```

Concentriamoci ora su assembler:

```

.data
.LC0:
.ascii "Hello World! I welcome you %d times.\n"

.text
main:
    pushq %rbp
    movq %rsp, %rbp
    subq $8, %rsp
    movl $100, -4(%rbp)
    movl -4(%rbp), %esi
    leaq .LC0(%rip), %rdi
    xorl %eax, %eax
    call printf@PLT
    xorl %eax, %eax
    addq $8, %rsp
    leave
    ret
    
```

direttive assembly
etichette
variabili globali
opcode
registri destinazione
registri sorgente
costanti
operandi in memoria

Quindi in generale un programma assembler viene implementato così :

```

.org [INDIRIZZO CARICAMENTO]
.data
    # Dichiarazione costanti e variabili globali

.text
main:
    # Corpo del programma
    hlt # Per arrestare l'esecuzione
    
```

Vediamo ora le direttive assembly :

1. **Label** : mnemonico testuale definito dal programmatore ed associato alla sequenza di istruzioni immediatamente seguenti
2. **Location counter**: viene identificato da "." : rappresenta il valore dell'indirizzo corrente; può essere usato per far saltare la "generazione di indirizzi" magando usandolo per calcolare le dimensioni delle strutture dati:


```

msg:
a. .ascii "Hello, world!\n"

len = . - msg
            
```
3. **.org (indirizzo), fill**: metodo alternativo per impostare il location counter, impostando i byte a fill
4. **.equ symbol, expression** : definisce una costante (non occupa memoria al momento della dichiarazione) ;
 - a. Alternativa possibile : symbol=expression
 - b. Possibile riusare il simbolo in più parti del codice, ma non prima della sua definizione.
5. **.byte expression**: riserva in memoria (byte) per espressione
 - a. var: .byte 0
 - a. array: .byte 0, 1, 2, 3, 4, 5
6. **.word** : analogo a .byte ma riserva memoria a 2 byte(16 bit)
7. **.longword** : analogo a .bye ma riserva memoria a 4 byte(32 bit)
8. **.quadword** : analogo a .bye ma riserva memoria a 8 byte(64 bit)
9. **.ascii "string"**: riserva in memoria per un vettore di caratteri ed imposta il valore a string
10. **.fill repeat, size**: riserva una regione di memoria composta da repeat celle di dimensione size impostata a value (default size=1 value=0).

11. **.text** : da qui in poi va nella sezione testo
12. **.data** : da qui in poi va nella sezione data
13. **.comm symbol, length**: dichiara un'aria di memoria con nome (symbol) di dimensione (length) nella sezione .bss
14. **.driver ivn**: identifica l'inizio della routine del servizio al codice ivn.

Andiamo ora a vedere le **istruzioni di classe 6 : le istruzioni di controllo del flusso** : permettono di specificare se e quando ed in quale ordine devono essere eseguite le istruzioni che compongono il sorgente : nel caso di C è molto semplice in quanto vi sono poche strutture di controllo , ma in assembler leggermente più "difficile": in quanto servono controlli di condizione , iterazioni e salti a funzioni (sotto-programmi). Vediamo un esempio :

<pre> .org 0x800 .data message: .ascii "Hello world!" counter: .byte 0 .text main: movq \$message,%rax .repeat: cmpb \$0, (%rax) jz .end addb \$1, counter addq \$1, %rax jmp .repeat .end: hlt </pre>	Questo programma conta la lunghezza della stringa passata in input, usando un confronto (cmpb-> COMPARE BIT) ed un salto .jz(jump zero -> salta se zero) e jmp (salto "semplice).
---	--

Andiamo a vedere come funziona in dettaglio l'istruzione di compare (CMP): unica istruzione che allineandosi con il bit di stato , permette di valutare una condizione . In dettaglio :

CMP B,E	B e E sono operandi . Confronto tra il valore di B ed E , calcolando B-E , e il risultato viene scartato.
----------------	--

Vediamo un esempio :

```

if(x == 1) {
    CODE BLOCK A;
} else if(x == 2) {
    CODE BLOCK B;
} else {
    CODE BLOCK C;
}

if(TEST)
    SINGOLO STATEMENT;
else
    SINGOLO STATEMENT;

```

```

    cmpb $1, %a1
    jnz .elseif
    # CODE BLOCK A
    jmp .endif
    .elseif:
    cmpb $2, %a1
    jnz .else
    # CODE BLOCK B
    jmp .endif
    .else:
    # CODE BLOCK C
    .endif:

```

Andiamo nel dettaglio : come già detto il confronto tra due variabili, si basa su una sottrazione tra i due operandi , aggiornando gli opportuni bit del registro flags : andiamo a vedere un esempio :

A-B --> A-B=0

1. Se $a-b > 0$ il risultato va bene , senza prestito aggiuntivo
2. Se $a-b < 0$, allora si deve "prestare un'unità a b"
3. **In generale quindi : nel caso di sottrazione la CU rileva la necessità di prestito verificando se addizione in complemento a 2 genera un riporto .**
 - a. Se nell'addizione **non** c'è un riporto , allora **non** c'è prestito nella sottrazione
 - b. Se nella c'è un riporto, allora non c'è prestito nella sottrazione.
4. Se nel caso invece di sub , il CF viene negato .

Andiamo a vedere i tipi di confronti , ovvero quelli tra **operandi senza segno** e quelli tra **operandi con segno** . Andiamo a vedere i primi : si assume che entrambi gli operandi siano ≥ 0 , quindi si deve controllare il valore di CF e ZF per determinare l'ordine . Quindi in modo più generale una generica istruzione **CMPX src,dst** , dove la "X" rappresenta il tipo di salto , si ha che :

Condizione	Primo controllo	Secondo controllo
$dest < source$	CF = 1	
$dest \geq source$	CF = 0	
$dest > source$	CF = 0	ZF = 0
$dest = source$	ZF = 1	
$dest \neq source$	ZF = 0	

Vediamo ora due esempi : il primo vediamo se $2 > 1$ (il che è ovvio), ma andiamo a vedere il dettaglio :

CMP \$2, \$1 → $0001 - 0010$

$\begin{array}{r} 0001 \\ - 0010 \\ \hline 1111 \end{array}$

ma in realtà la CU opera

aritmetica in CP2 → $1 + (-2)$

$\begin{array}{r} 0001 \\ + 1110 \\ \hline 1111 \end{array}$

→ $-1 < 2$

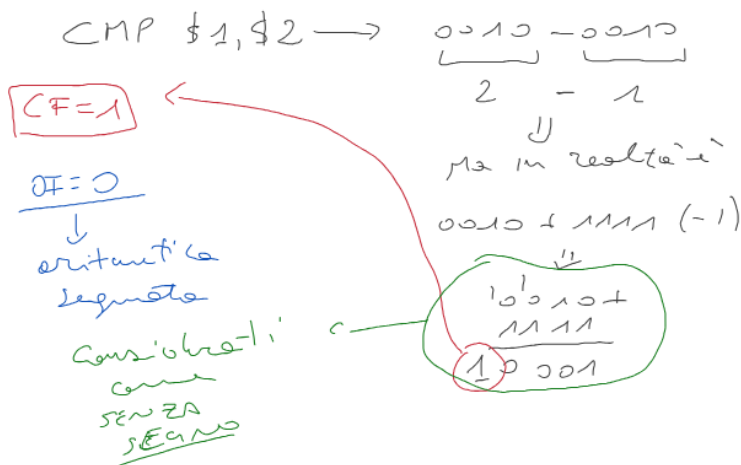
CF = 0

ZF = 0

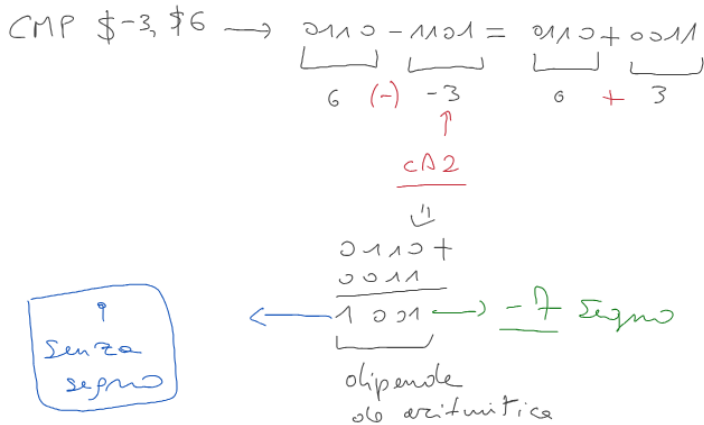
⇒ $2 > 1$

CF = 0

Invece il secondo esempio : $1 > 2$ (la condizione fallisce):



Vediamo ora i secondi : quelli in aritmetica segnata : da notare che CF in questo caso non ha in significato preciso in aritmetica a complemento a 2, in quanto si vedono solo gli ultimi 2 riporti. Quindi in generale facciamo **$dst - src < 0$** : poniamo attenzione a SF (sign flag). Vediamo un esempio :



Da notare che se consideriamo solo quella segnata : **si ha che la somma è 9, ma in CA2 non può essere rappresentato, quindi vi è stato overflow**. Riassumendo quindi :

Condizione	Primo controllo	Secondo controllo
$dest < source$	$SF \neq OF$	
$dest \geq source$	$SF = OF$	
$dest > source$	$ZF = 0$	$SF = OF$
$dest = source$	$ZF = 1$	
$dest \neq source$	$ZF = 0$	

Vediamo ora dal punto di vista dell'assembler:

```

.org 0x800
.data
x: .word 3
y: .word -2
.text
# Imposta a 1 l'indirizzo 0x1280 solo se x > y
# Assumo che x ed y possano assumere valori negativi
movw x, %ax
movw y, %bx
cmpw %bx, %ax
jz .nonImpostare
js .SFset
jo .nonImpostare # eseguita se SF = 0. Se OF = 1 allora SF != OF
jmp .set
.SFset:
jno .dont # eseguita se SF = 1. Se OF = 0 allora SF != OF
.set:
movb $1, 0x1280
.dont:
hlt

```

Dopo questa intro, andiamo ora a vedere la classe 6 : **attenzione al tipo di operando coinvolti** . Questa classe si occupa delle istruzioni di controllo condizionale del flusso : in base ad una certa condizione nel programma, devo poter alterare il flusso del programma. Per fare questa alterazione del flusso , **si devono vedere i bit nel registro flag** . Vediamo ora tutti i tipi di salto :

Istruzione	Descrizione	Aritmetica	Condizione controllata
jnb	Jump if below	non segnata	$CF = 1$
jnae	Jump if not above or equal		
jnb	Jump if not below	non segnata	$CF = 0$
jae	Jump if above or equal		
jbe	Jump if below or equal	non segnata	$CF = 1$ o $ZF = 1$
jna	Jump if not above		
ja	Jump if above	non segnata	$CF = 0$ e $ZF = 0$
jnb	Jump if not below or equal		
jl	Jump if less	segnata	$SF \neq OF$
jnge	Jump if not greater or equal		
jge	Jump if greater or equal	segnata	$SF = OF$
jnl	Jump if not less		
jle	Jump if less or equal	segnata	$ZF = 1$ o $SF \neq OF$
jng	Jump if not greater		
jg	Jump if greater	segnata	$ZF = 0$ e $SF = OF$
jnl	Jump if not less or equal		

Andiamo a vedere ora il codice per implementare i salti :

1. Switch :

a.	<pre>switch (OPERAND) { case CONSTANT: CODE; break; default: CODE; }</pre>	<pre>movl var, %eax shll \$3, %eax movq branchTable(%rax), %rax jmp *%rax</pre>
----	--	---

b. In generale :

```
switch (age) {
  case 1: printf("You're one.");      break;
  case 2: printf("You're two.");     break;
  case 3: printf("You're three.");   __attribute__((fallthrough));
  case 4: printf("You're three or four."); break;
  default: printf("You're not 1, 2, 3 nor 4!");
}
```

2. While :

a.

```
while(TEST) {
  CODE;
}

while(TEST) {
  if(OTHER_TEST) {
    continue;
  }
  CODE;
}
```

b.

```
while(TEST) {
  if(OTHER_TEST) {
    break;
  }
  CODE;
}

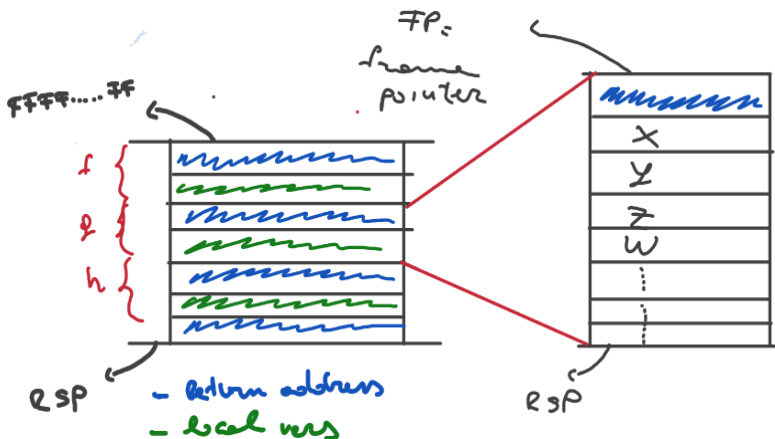
.test:
  cmpb ...
  jnz .skip
  # <codice>
  jmp .test
.skip:
```

3. Do/while - for :

	<pre>do { CODE; } while(TEST);</pre>	<pre>for(INIT; TEST; POST) { CODE; }</pre>
a.	<pre>.begin: # <codice> cmpb ... jz .begin</pre>	<pre>movq \$0, %rcx movq \$3, %rbx .test: cmpq %rbx, %rcx jz .end # <codice> addq \$1, %rcx jmp .test .end:</pre>

4. Salti e chiamate a funzioni : da notare che il goto è equivalente di jmp ; se in presenza di goto o jmp si ha una chiamata di istruzione CALL.

Concentriamoci ora sul C : ogni variabile in questo linguaggio ha associato un tipo che la caratterizza , e questi tipi sono raggruppati in 3 categorie : **primitivi, aggregati e puntatori** . Solo i primi due hanno una corrispondenza in assembler ed in più i secondi vengono convertiti in accesso ai tipi primitivi . Ognuna delle variabili ha due contesti (scope) di ambito e di classe : le prime (**anche dette globali**), risiedono all'interno delle sezioni .data e .bss , mentre le locali (**anche dette automatiche**), occupano memoria all'interno dello stack , mentre le ultime (**statiche**) sono delle locali, ma ristrette all'interno della funzione/modulo . Per una qualunque funzione (almeno in C), si ha un **record di attivazione** : zona di memoria che contiene i parametri formali e le variabili locali della funzione stessa. Non ha una dimensione fissa, in quanto dipende dal tipo della funzione (parametri). Quando la funzione termina, il record di attivazione viene cancellato dallo stack , **ovvero invalidato logicamente**, liberando così la memoria per successivi record di attivazione. Vediamo logicamente come è fatto :



Notiamo che RSP (Register stack pointer) punta alla fine della memoria, quindi all'inizio dello stack. Mentre se si parla di FP(frame pointer) indica la base del record di attivazione (FFFF...FFFF). Se si ha una sola variabile accedere all'indirizzo è facile , uso l'indirizzo della prima cella di RSP , mentre se devo prendere l'n-esima cella, devo spazzarmi dentro RSP. Vediamo esempio : ho x e y variabili , andiamo a caricare l'indirizzo:

X	Y
movl (%rsp),%eax	Movl 4(%rsp),%eax

Iterando il procedimento funzionerebbe, ma si potrebbe perdere il conto della cella, quindi optiamo per una soluzione simile: stesso principio base, ma salviamo la base dello stack pointer in un registro (rsp -> rbp): Register base pointer, il quale viene decrementato in base al numero di celle che servono:

```

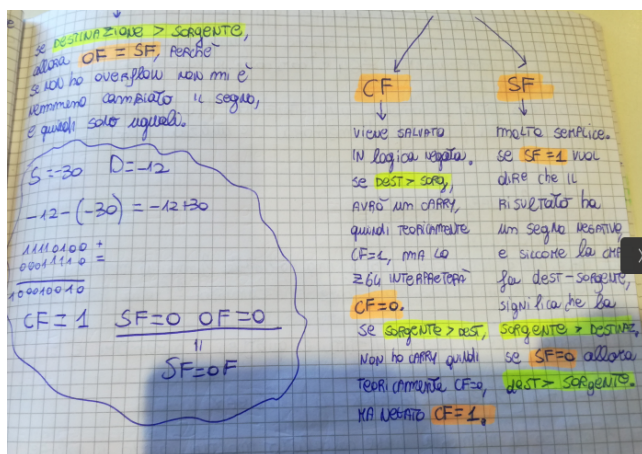
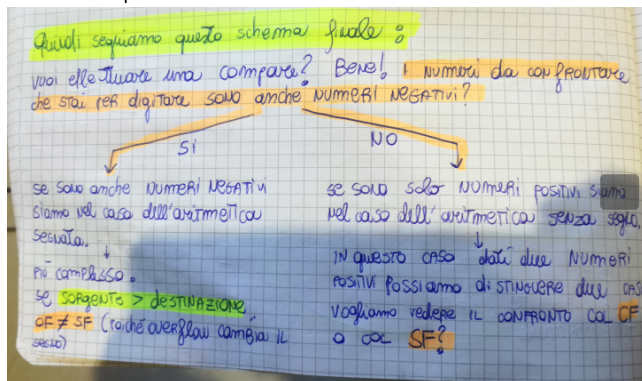
void function() {
    int x = 128;
    ...
    return;
}

function:
    pushq %rbp
    movq %rsp, %rbp
    subq $4, %rsp
    movl $128, -4(%rbp)
    ...
    addq $8, %rsp
    leave
    ret

```

Le prime 3 istruzioni (pushq, movq, subq) sono intese come preambolo, mentre le ultime 3 sono considerate come tail (rispristino stack e ritorno ad indirizzo del chiamante +1).

Riassumendo quindi:

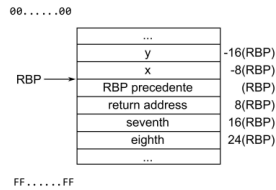


Andiamo ora a vedere come vengono passati i parametri nelle funzioni: **calling convention**. Quindi si parla di modalità di uso dei registri per passare i parametri: in generale sono "limitati", quindi si opta per passaggio tramite stack, registri + stack e misto. Il valore di ritorno anche lui viene salvato in un registro general purpose. In dettaglio i primi 6 registri servono per passare i 6 parametri; gli altri parametri rimanenti devono essere messi sullo stack, ma in ordine inverso rispetto a come compaiono, mentre per quelli di ritorno RAX e RDX. Quindi i registri si dividono in due **callee-save** (salvati dal chiamato), per ripristino prima del ritorno al chiamante: RBP, RBX, R12-R15, e **caller-save**: tutti gli altri. In dettaglio:

- I primi sei parametri (interi e indirizzi) di una subroutine vengono passati tramite registri:
 - RDI, RSI, RDX, RCX, R8, R9
- Se una subroutine accetta più di sei parametri, si utilizza lo stack per quelli aggiuntivi
- Si utilizzano due registri per il valore di ritorno:
 - RAX e RDX
- I registri sono divisi in *callee save* e *caller save*
 - callee-save: RBP, RBX, R12-R15
 - caller-save: tutti gli altri

Attenzione che il valore massimo di ritorno è 128 bit, quindi devo "spezzare" il dato usando due registri a 64 bit . Vediamo un esempio :

```
void f(int first, int second, int third, int fourth, int fifth,
      int sixth, int seventh, int eighth, int ninth, int tenth,
      int x, y;
      ...
}
```



X e Y sono variabili globali. Il chiamante li setta : i primo 6 parametri nei parametri secondo calling convention . Mentre il settimo e l'ottavo sono invertiti nello stack, in quanto ogni elemento rappresenta una distanza dalla base via via crescente . Nella cella più bassa dello stack , vi è l'indirizzo dell'istruzione successiva della CALL (fatto dalla ret) , il quale rappresenta l'indirizzo di RIP durante la fase di fetch. Andiamo a vedere ora i tipi di dato in C: ci sono 4 tipi di dato : **char** (quantità minima di memoria che il processore indirizza) , **int** , **float** , **double**. Vediamo ora come definire in modo univoco le grandezze dei dati, evitando incomprensioni tra sistemi e processori : usando altri tipi di dato , ma anche così' non riesco a bypassare : **signed** , **unsigned** , **short** e **long**. Attenzione al tipo void : la funzione non restituisce nulla . In dettaglio :

Tipo	Spiegazione	Dimensione minima (bit)	Specificatore di formato
char	Minima unità indirizzabile della macchina che contiene l'insieme basilare di caratteri. È di tipo intero. Può essere segnato o non segnato. Contiene CHAR_BIT bit.	8	%c
signed char	Come un char, ma segnato. Contiene almeno l'intervallo [-127, +127].	8	%c (%hhi per il numero)
unsigned char	Come un char, ma non segnato. Contiene almeno l'intervallo [0, 255].	8	%c (%hhu per il numero)
short short int signed short signed short int	Tipo intero segnato breve. Contiene almeno l'intervallo [-32,767, +32,767].	16	%hi o %hd
unsigned short unsigned short int	Tipo intero non segnato breve. Contiene almeno l'intervallo [0, 65,535].	16	%hu
int signed int signed int	Tipo intero segnato di base. Contiene almeno l'intervallo [-32,767, +32,767]	16	%i o %d

perché convenzionalmente lo usiamo come se fosse a 32 bit?

Tipo	Spiegazione	Dimensione minima (bit)	Specificatore di formato
unsigned unsigned int	Tipo intero non segnato di base. Contiene almeno l'intervallo [0, 65,535].	16	%u
long long int signed long signed long int	Tipo intero segnato lungo. Contiene almeno l'intervallo [-2,147,483,647, +2,147,483,647].	32	%li o %ld
unsigned long unsigned long int	Tipo intero non segnato lungo. Contiene almeno l'intervallo [0, 4,294,967,295]	32	%lu
long long long long int signed long long signed long long int	Tipo intero segnato lungo lungo. Contiene almeno l'intervallo [-9,223,372,036,854,775,807, +9,223,372,036,854,775,807] (dal C99)	64	%lli o %lld
unsigned long long unsigned long long int	Tipo intero non segnato lungo lungo. Contiene almeno l'intervallo [0, +18,446,744,073,709,551,615] (dal C99)	64	%llu
float	Tipo reale a singola precisione. Le sue proprietà non sono specificate dallo standard.		%f
double	Tipo reale a doppia precisione. Le sue proprietà non sono specificate dallo standard.		%lf o %f
long double	Tipo reale a precisione estesa. Le sue proprietà non sono specificate dallo standard.		%Lf o %LF

Attenzione comunque ai **warning** : chiedo al compilatore di farmi vedere i warning : -Wall -Wextra. Inoltre è possibile passare da un tipo di dato all'altro : si parla di **casting** . Vediamo un esempio :

<pre>#include <stdio.h> int main(int argc, char *argv[]) { double result=1/2*2; printf("result is : %lf\n", result); //valore in virgola mobile double result1 = 1. / 2 * 2; printf("Result1 is %lf\n", result1); double result2 = (double)1 / 2 * 2; printf("Result2 is %lf\n", result2); double result3 = 1 / 2 * (double)2; printf("Result3 is %lf\n", result3); return 0; }</pre>	<pre>result is : 0.000000 Result1 is 1.000000 Result2 is 1.000000 Result3 is 0.000000</pre>
---	---

Inoltre le classi in C , possono avere degli identificatori :

- auto**: allocazione automatica (fondamentalmente inutile)
- register**: allocazione automatica. Suggerisce anche al compilatore di inserire l'oggetto in un registro della CPU (obsoleto)
- static**: linking interno: non è consentito accedere all'oggetto dall'esterno del modulo C o della funzione
- extern**: linking esterno: si indica al compilatore che l'oggetto è stato/sarà definito in un altro modulo C, affidando al linker il compito di rilocallo (ridondante)

Se vogliamo avere una rappresentazione univoca (importando la libreria stdin.h) :

Tipo	Definizione
int8_t	Intero segnato a 8 bit
uint8_t	Intero non segnato a 8 bit
int16_t	Intero segnato a 16 bit
uint16_t	Intero non segnato a 16 bit
int32_t	Intero segnato a 32 bit
uint32_t	Intero non segnato a 32 bit
int64_t	Intero segnato a 64 bit
uint64_t	Intero non segnato a 64 bit

Andiamo a fare un riepilogo attraverso un esempio : file if.c

```
#include<stdio.h>
int main(int argc,char *argv[])
{
    int i=0;
    if(argc==1)
    {
        printf("you have only one argument \n");
        printf("parameter @ position [%d] %s \n",i,argv[i]);
    }else if(argc>1)
    {
        printf("Here's your argument :\n");
        for (;i<argc;i++)
        {
            printf("parameter @ position [%d] %s \n",i,argv[i]);
        }
        printf("\n");
    }
    return 0;
}
```

```
daniele@daniele-Aspire-E5-571G:~/Scrivania/programmi_calcolatori/esercizi_c$ gcc -Wall -Wextra if.c -o if
daniele@daniele-Aspire-E5-571G:~/Scrivania/programmi_calcolatori/esercizi_c$ ./if
you have only one argument
parameter @ position [0] ./if

daniele@daniele-Aspire-E5-571G:~/Scrivania/programmi_calcolatori/esercizi_c$ ./if first second third fourth fifth
Here's your argument :
parameter @ position [0] ./if
parameter @ position [1] first
parameter @ position [2] second
parameter @ position [3] third
parameter @ position [4] fourth
parameter @ position [5] fifth
```

Andiamo a parlare delle **variabili booleane** : variabili che valgono 0/1->false/true. Questo tipo di dato non esiste , o meglio nessuna isa implementa questi che li gestisce. In C per gestire questo tipo di dato si usa un **header : stdbool.h** , il quale header definisce parole chiave tipo **true**, **false**, **bool**. In assembler questi confronti vengono implementati dalla JZ. Facciamo una bella ripassata degli operatori in generale : logici, aritmetici, ecc ecc :

Operatori Aritmetici	Operatori di Confronto	Assegnazione
+	== uguaglianza	= assegnazione uguale
-	!= non uguaglianza	+= assegnazione più-uguale
*	> maggiore di	-= assegnazione meno-uguale
/	< minore di	*= assegnazione per-uguale
%	>= maggiore o uguale	/= assegnazione diviso-uguale
++ incremento	<= minore o uguale	%= assegnazione modulo-uguale
-- decremento		<<= assegnazione shift-sinistro-uguale
Operatori Logici	Operatori Bit a Bit	>>= assegnazione shift-destro-uguale
&& and logico	& and binario	&= assegnazione and-uguale
or logico	or binario	^= assegnazione xor-uguale
! negazione logica	^ xor binario	= assegnazione or-uguale
? : confronto ternario	~ complemento a 1	
	<< shift a sinistra	
	>> shift a destra	

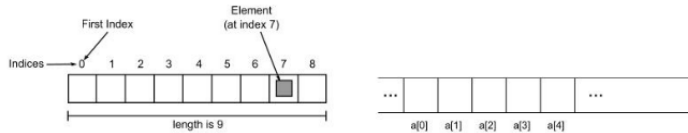
Vediamo un esempio :

```
#include <stdio.h>
int main(int argc,char *argv[])
{
    int i=26;
    do{
        /*
        decremento
        da notare il momento in cui voglio accedere
        in questo caso POST-DECREMENTO
        prima leggo e poi decremento
        */
        printf("%d, ",--i);
    }while(i);
    /*
    il cursore arretra di due caratteri
    quindi si ha la sovrapposizione dei caratteri
    */
    printf("\b\b \n");
    i=0;
    while(i<26)
    {
        printf("%d",i);
        i++;
    }
    printf("\b\b \n");
    return 0;
}
```

```
daniele@daniele-Aspire-E5-571G:~/Scrivania/programmi_calcolatori/esercizi_c$ gcc bool_while.c -o bool
daniele@daniele-Aspire-E5-571G:~/Scrivania/programmi_calcolatori/esercizi_c$ ./bool
25, 24, 23, 22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0
0123456789101112131415161718192021222324 5
```

Andiamo a vedere un'altra possibile collezione di elementi : **gli array** . Questi elementi in memoria , vengono rappresentati in modo contiguo (vedasi memoria flat) . In base al tipo di dato che deve

immagazzinare, si ha una differente taglia del dato; inoltre ricordiamoci l'architettura little endian.



Quindi un classico esempio di vettore è il seguente, dove le parentesi "[]" (spiazzamento) stanno ad indicare il numero di byte che mi devo spostare, sempre in relazione al tipo di dato :

$$A[i]=a + size*i$$

Dove size rappresenta la taglia dell'oggetto (ciascun elemento), mentre i rappresenta il numero di elementi di cui mi voglio spostare.

Vediamolo ora in assembly :

```
movq $0, %rcx # %rcx viene usato come indice
movq $array, %rax # %rax viene usato come base
loop:
movq (%rax, %rcx, 8), %rdx # sposta i dati dove serve
# <processa i dati>
addq $1, %rcx
cmpq $size, %rcx
jnz .loop
```

Da notare che in %rax si ha l'indirizzo di base del vettore; nell'istruzione di spiazzamento si accede in memoria , per poi farci qualunque cosa. Poi per aggiornare l'indice aggiungo 1 al contatore , che in memoria ed essendo/operando con quadword, incremento di 8!!

In alternativa a questo :

```
movq $0, %rcx # %rcx viene usato come indice
loop:
movq array(, %rcx, 8), %rdx # sposta i dati dove serve
# <processa i dati>
addq $1, %rcx
cmpq $size, %rcx
jnz .loop
```

In quanto si usa spiazzamento con operandi nel bss : indirizzi vicini alla sezione dati : risparmio un registro base !!

Altro esempio:

```
movq $array, %rax # carica indirizzo del primo elemento
movq $size, %rcx
oop:
movq (%rax), %rdx # sposta i dati dove serve
# <processa i dati>
addq $24, %rax
subq $1, %rcx
jnz .loop
```

Attenzione se elemento più grande di quadword (elementi di scala >8) : in questo caso devo incrementare del numero di byte che mi serve , a patto che parto da indirizzo di base. **Notiamo che a posto di cmp usiamo la sub : bit di flag testato dentro qualunque operazione** .Andiamo a vedere ora un'altra rappresentazione in memoria : **le Stringhe:**

Overo una rappresentazione in memoria di caratteri ; non si conosce a priori la memoria , quindi alla fine si inserisce il **terminatore di stringa** ('\0'), per evitare undefined behaviour . Vediamo un esempio :



Andiamo a vedere un esempio pratico :

```
#include <stdio.h>
int main(int argc, char *argv[])
{
    int nums[4]={0};
    char name[4]='a','p';

    /*
    qui nel primo caso , del vettore di interi
    stampa tutto il vettore inizializzato a 0;
    Nel secondo caso invece , anche avendo un vettore di 4 elementi
    ne stampa solo 2 : gli altri due sono i terminatori di stringa ;
    Nel terzo caso output simile , perchè il compilatore va in memoria
    e scorre tutto l'array finché non trova il primo terminatore
    */
    printf("nums : %d%d%d%d\n",nums[0],nums[1],nums[2],nums[3]);
    printf("name each : %c%c%c%c \n",name[0],name[1],name[2],name[3]);
    printf("name: %s\n",name);
}
```

```
nums : 0000
name each :ap
name: ap
```

Inizializziamo ora i vettori :

<pre>#include <stdio.h> int main(int argc, char *argv[]) { int nums[4]={0}; char name[4]={'a','p'}; /* qui nel primo caso , del vettore di interi; stampa tutto il vettore inizializzato a 0; Nel secondo caso invece , anche avendo un vettore di 4 elementi ne stampa solo 2 : gli altri due sono i terminatori di stringa ; Nel terzo caso outpu simile , perchè il compilatore va in memoria e scorre tutto l'array finché non trova il primo terminatore */ printf("nums : %d%d%d%d\n",nums[0],nums[1],nums[2],nums[3]); printf("name each :%c%c%c%c \n",name[0],name[1],name[2],name[3]); printf("name: %s\n",name); /* andiamo ora ad "popolare" i vettori e ristampiamo i valori */ nums[0] = 1; nums[1] = 2; nums[2] = 3; nums[3] = 4; name[0] = 'A'; name[1] = 'l'; name[2] = 'e'; name[3] = '\0'; printf("nums: %d %d %d %d\n", nums[0], nums[1], nums[2], nums[3]); printf("name each: %c %c %c %c\n", name[0], name[1], name[2], name[3]); printf("name: %s\n", name); }</pre>	<pre>nums : 0000 name each :ap name: ap nums: 1 2 3 4 name each: A l e name: Ale</pre>
---	--

Attenzione che questo programma è logicamente corretto , ma è sbagliato in quanto questa è prima invocazione di funzione : **lo stack è vuoto. In determinate circostanze può portare ad undefined behaviour.** Torniamo al discorso della lunghezza di un tipo o di una variabile (restituito in multipli di char) : **SIZEOF : nel caso di char vale sempre 1**

Vediamo ora come codificare elementi e mapparli su un determinato tipo : **enum** : ci possiamo anche differenziare i vari tipi di casi (come per esempio switch), ed assegnargli un valore numerico:

```
#include<stdio.h>

/*
il compilatore in base al numero degli elementi
comincia ad assegnargli un valore crescente
partendo da 0
*/
enum suit{
    CLUB=0,
    DIAMONDS=1,
    HEARTS=2,
    SPADES=3
};

//qui non passiamo argomenti : argc=0 e argv nullo
int main (void)
{
    card = CLUB;
    // se stampiamo il valore con %d il compilatore da warning
    // stampa il numero di interi
    printf("Size of enum variable = %ld bytes\n", sizeof(card));
    return 0;
}
```

Size of enum variable = 4 bytes

Vediamo ora come combinare entrambi i concetti : creiamo un array dinamico : **array a lunghezza variabile** : istanziati su stack a tempo di compilazione:

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>

//sequenza di numeri pseudo casuali
//compeso tra 0 e max (rand(max))
void generate_and_print(int n)
{
    int i;
    double vals[n];
    for (i=0;i<n;i++)
    {
        vals[i]=(double)rand()/RAND_MAX*50;
    }
    for (i=0;i<n;i++)
    {
        //stampo il double (precisione singola)
        //mi fermo a 3 cifre decimali
        printf("%.03f",vals[i]);
    }
    printf("\b\b\n");
}

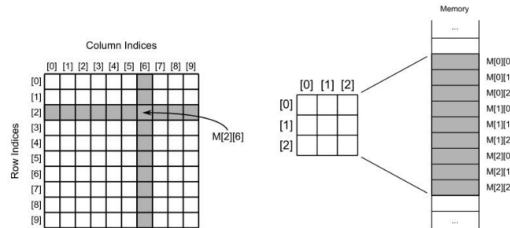
int main(void)
{
    srand(time(NULL));
    generate_and_print(3);
    generate_and_print(12);
    generate_and_print(1);
}
```

```

4.9296.5401.358
2.07623.05122.27640.58022.1118.25615.2815.02811.7557.00022.5883.229
8.872

```

Andiamo a vedere un'altra rappresentazione dei dati : **matrici** , ovvero strutture dati a più dimensioni :
ma dato il modello di memoria flat , lo dobbiamo linearizzare . Quindi si deve memorizzare linea e/o
colonna in modo contiguo. Quindi per accedere allo spiazzamento ([[]]) si usa la seguente formula :
address $\iff i * N * size + j * size$, dove n è il numero di elementi nella riga. Vediamo concettualmente
come sono organizzate :



Vediamo ora come passare una matrice come parametro :

Handwritten notes:
 $f(m[i][j]) \rightarrow \text{int } n[i][j]$ *però ho fatto così*
RSI
Es: $f(n[i][j])$
 $\{ n[i][0] : i; \}$
Side effect
però alla funzione
una copia del
valore!
il chiamante
vedrà un
valore diverso
Quindi: il passaggio
dei parametri viene
la ho per valore

Mentre dal punto di vista del codice :

```

#include<stdio.h>
/*
notiamo che la prima matrice viene dichiarata normalmente
nella seconda invece il numero delle righe lo calcola il compilatore
nell'ultimo caso dichiariamo una matrice vuota
*/
int A[2][3]={{1,3,0},{-1,5,9}};
int B[1][3]={{2,7,-4},{3,-2,7}};
int C[2][3]={{0,0,0},{0,0,0}};

void print_matrix(int m[2][3])
{
    int i,j;
    for(i=0;i<2;i++)
    {
        for(j=0;j<3;j++)
        {
            printf("%d\t",m[i][j]);
        }
        puts("");
    }
}

int main(int argc, char *argv[])
{
    int i, j;
    puts("Matrix A:");
    print_matrix(A);
    puts("\nMatrix B:");
    print_matrix(B);
    for(i = 0; i < 2; i++) {
        for(j = 0; j < 3; j++) {
            C[i][j] = A[i][j] + B[i][j];
        }
    }
    // stampa matrice C come la somma delle due
    puts("\nMatrix C:");
    print_matrix(C);
}

```

```

Matrix A:
1   3   0
-1  5   9

Matrix B:
2   7  -4
3  -2   7

Matrix C:
3  10  -4
2   3  16

```

```

#include <stdio.h>
/*
esempio di matrice a dimensione variabile
*/
int A[2][3] = {{1, 3, 0}, {-1, 5, 9}};
int B[1][3] = {{2, 7, -4}, {3, -2, 7}};

void print_matrix1(int row, int col, int M[row][col])
{
    int i, j;
    for(i = 0; i < row; i++) {
        for(j = 0; j < col; j++) {
            printf("%d\t", M[i][j]);
        }
        puts("");
    }
}

void print_matrix2(int col, int M[][col])
{
    int i, j;
    for(i = 0; i < 2; i++) {
        for(j = 0; j < col; j++) {
            printf("%d\t", M[i][j]);
        }
        puts("");
    }
}

int main(int argc, char *argv[])
{
    /*
    qui è il compilatore/assemblatore che calcola
    l'indice dello spiazzamento
    */
    puts("Matrix A:");
    print_matrix1(2, 3, A);
    puts("\nMatrix B:");
    print_matrix2(3, B);
    return 0;
}

```

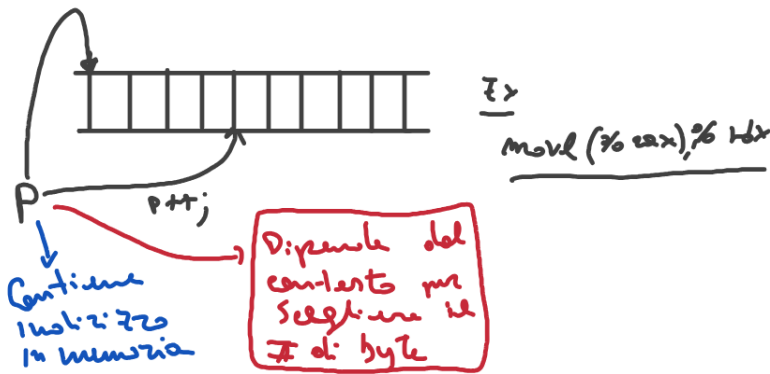
```

Matrix A:
1   3   0  -1   5   9
Matrix B:
2   7  -4   3  -2   7

```

Ricordiamo quindi che l'importante è che il numero di colonne sia definito, in quanto quello delle
righe lo calcola il compilatore.

Andiamo ora a vedere la bestia nera del C: i **puntatori**. Queste variabili non contengono un valore, ma un indirizzo di memoria. In dettaglio:



In dettaglio (vediamolo in C):

```
int *ptr; // puntatore a cella che contiene intero
```

```
int var1, var2; // var2 non è pointer;
```

puntatore a cella in memoria che contiene int
variabile di tipo int

*ptr → indirizzo
↓
contenuto

Nota: * non è moltiplicatore di tipo ma di variabile

void* → generico puntatore: indir. puntato dal # di byte

Vediamo ora come è possibile "giocarci": ovvero l'**aritmetica dei puntatori**:

type *ptr: Un puntatore di tipo type chiamato ptr

*ptr: Il valore della variabile puntata da ptr, qualunque sia il tipo

*(ptr + i): Il valore di (qualsiasi cosa si trovi all'indirizzo ptr + i)

&var: L'indirizzo della variabile var

type *ptr = &var: Un puntatore di tipo type chiamato ptr inizializzato all'indirizzo di var

ptr++: Incrementa l'indirizzo cui punta ptr

Nota: si usa solo la base, in quanto indice non ha senso. Esempio:

* (ptr + i) → spostamento di i-cella
sull'orizzonte del valore
↓
(%rax, %rcx, ??)
l'indice non c'è

&var: l'indirizzo in memoria della variabile
(lea vs mov)

Type *ptr = &var

ptr++; incremento indirizzo di 1!
Dipende dal contesto!

Come vengono usati i puntatori in assembler? Si usa un registro per contenere indirizzo di memoria , quel registro viene usato come registro base ed il contenuto può essere manipolato a piacimento :

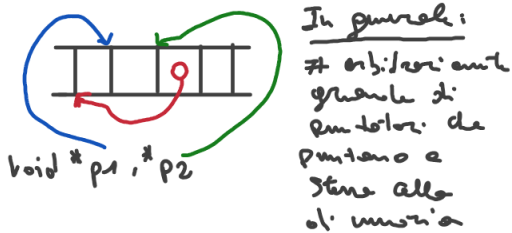
```
int *intptr;          movq $intptr, %rax
*intptr = 42;         movl $42, (%rax)

int *ptr = &var;      movq $var, %rax
                    movq %rax, ptr

ptr++;                addq $4, %rax
```

Nota : aggiungo costante 4 al registro perché stiamo usando interi.

Facciamo un discorso più generale :



Vediamo l'aritmetica:

Come detto in precedenza:

- ptr++: Incrementa l'indirizzo cui punta ptr

I puntatori tuttavia hanno un tipo, pertanto cosa vuol dire "incrementare l'indirizzo?"

L'aritmetica dei puntatori è tale per cui l'indirizzo contenuto nella variabile puntatore viene incrementato di un numero di byte pari alla taglia del tipo primitivo puntato

dato un `int *p = (void *)1000`, assumendo `sizeof(int) == 4`:

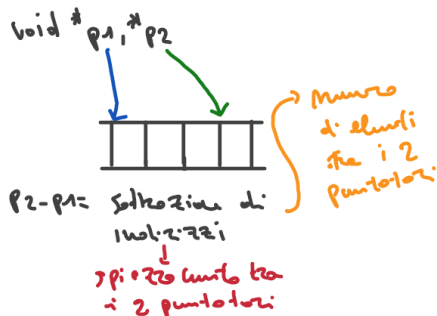
- `p+1 == 1004`
- `p+2 == 1008`
- `p+n == 1000+n*4`

Operazioni valide sono:

- incremento, decremento, somma, sottrazione
- sottrazione tra puntatori: determina il numero di elementi tra gli indirizzi

Inoltre è possibile fare la sottrazione tra puntatori : ovvero la distanza in byte tra i due puntatori :

dimensione in byte :



Vediamo un esempio in C : stampo due vettori in quattro modi di differenza :

```
#include<stdio.h>
int main(int argc, char *argv[])
{
    // The data we will be printing
    int ages[] = { 23, 43, 12, 89, 2 };
    char *names[] = {
        "Alan", "Frank",
        "Mary", "John", "Lisa"
    };
    int count = 5;
    int i = 0;
    // first way: indexing
    for (i = 0; i < count; i++) {
        printf("%s has %d years alive.\n", names[i], ages[i]);
    }
    puts("----");
    // second way: using pointers
    int *cur_age = ages;
    /*
    qui uso il doppio puntatore
    in quanto le stringhe sono puntatori a caratteri ,
    ma in realtà si intendono come puntatori ad indirizzo
    che punta ad indirizzo
    */
    char **cur_name = names;
    for (i = 0; i < count; i++) {
        printf("%s is %d years old.\n", *(cur_name + i), *(cur_age + i));
    }
    puts("----");
    // third way: "offsetting" pointers
    for (i = 0; i < count; i++) {
        printf("%s is %d years old again.\n", cur_name[i], cur_age[i]);
    }
    puts("----");
    // fourth way: let's make it unnecessarily complex
    for (cur_name = names, cur_age = ages;
        (cur_age - ages) < count; cur_name++, cur_age++) {
        printf("%s lived %d years so far.\n", *cur_name, *cur_age);
    }
    return 0;
}
```

```
Alan has 23 years alive.
Frank has 43 years alive.
Mary has 12 years alive.
John has 89 years alive.
Lisa has 2 years alive.
---
Alan is 23 years old.
Frank is 43 years old.
Mary is 12 years old.
John is 89 years old.
Lisa is 2 years old.
---
Alan is 23 years old again.
Frank is 43 years old again.
Mary is 12 years old again.
John is 89 years old again.
Lisa is 2 years old again.
---
Alan lived 23 years so far.
Frank lived 43 years so far.
Mary lived 12 years so far.
John lived 89 years so far.
Lisa lived 2 years so far.
```

1) $14 \text{ v}[];$
 $14 * p : V;$
 $v[i] = *(p+i) \Leftrightarrow i[v]$

2) $\left[\begin{array}{l} \text{"p", "r", "p", "n"} \\ \text{char } *p; \end{array} \right]$ punteiro e
ind. 2, 2, 2
de punte
e ind. 2, 2, 2

```

GNU nano 4.8                                shift.c
#include<stdio.h>
#include<stdlib.h>
#include<limits.h>

#define MAX_BITS (int)((sizeof(unsigned int)*CHAR_BIT)
char string[MAX_BITS+1];
char *print_bits(unsigned int value);
char *print_bits(unsigned int value)
{
    int i;
    char *p;
    p = string + MAX_BITS;
    p[0] = 0;

    for(i = 0; i < MAX_BITS; i++) {
        *p++ = value & 1 ? '1' : '0';
        value >>= 1;
    }
    return string;
}

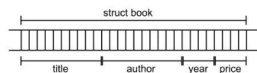
int main()
{
    unsigned int a=256, sb;
    int b=256,sb;
    scanf("%d",&a); // qui traslo a dx di 2 positon ( divido per 4)--> srl (shift right logica
    sb=a>>2; // qui traslo a dx di 2 positon ( divido per 4)--> srl (shift right logica
    printf("a[1]: %04d (%s)\n", a, print_bits(a));
    printf("after shift: %04d (%s)\n", sb, print_bits(sb));
    printf("b[1]: %04d (%s)\n", b, print_bits(b));
    printf("after shift: %04d (%s)\n", sb, print_bits(sb));
    return 0;
}

```

```
a          = 256 (00000000000000000000000000000000)
after shift = 064 (00000000000000000000000000000000)

b          = -256 (11111111111111111111111111111111)
after shift = -064 (11111111111111111111111111111111)
```

```
struct book {
    char title[10];
    char author[10];
    int publication_year;
    float price;
};
```



```
#include <stdio.h>
#include <assert.h>
#include <stdlib.h>
#include <string.h>

//qui si dichiara una struttura di tipo persona che contiene nome,eta,peso ed altezza

struct person{
    char *name;
    int age;
    int height;
    int weight;
};

struct person *person_create(struct person *who,char *name, int age, int height,int weight)
{
    //assert se persona non e nulla
    assert(who != NULL);
    who->name = strdup(name);
    who->age = age;
    who->height = height;
    who->weight = weight;
    return who;
}

void person_print(struct person *who)
{
    printf("Name: %s|Age: %d|Height: %d|Weight: %d\n",who->name,who->age,who->height, who->weight);
}

int main()
{
    struct person joe;
    (void)person_create(&joe, "Joe Alex", 32, 64, 140);
    printf("Joe is at memory location %p:\n", &joe);
    person_print(&joe);
    joe.age += 20;
    person_print(&joe);
}
```

```
Joe is at memory location 0x7ffc0093f080:  
Name: Joe Alex Age: 32 Height: 140 Weight: 140  
Name: Joe Alex Age: 52 Height: 140 Weight: 140
```

```
#include <stdio.h>
#include <stdlib.h>

struct student{
    char name[100];
    int roll;
    float marks;
};

// Funzione usata per stampare i dati
void print_structure(struct student *p)
{
    printf("Name :%s\tRoll: %d\tMarks :%f\n",p->name,p->roll,p->marks);
    printf("\n");
}

int main()
{
    //modo "1" per inizializzare la struct
    struct student stu1={"Marto",12,79.5};
    //modo "2" per inizializzare la struct
    struct student stu2={.roll=15,.name="Luigi",.marks=80.0};
    print_structure(&stu1);
    print_structure(&stu2);
}
```

```
Name :Mario      Roll: 12      Marks :79.500000
Name :Luigi     Roll: 15      Marks :80.000000
```

Analogamente a quanto detto per il C, andiamo ora a vedere le **struct** in assembler : per accedere ad ogni singolo campo della struttura , si usa la modalità di indirizzamento generale, ovvero base+ (indice)*spaziamento . In dettaglio : si calcola spaziamento a partire da indirizzo base della struct , si carica indirizzo iniziale della struct in un registro , si utilizza operando in memoria composto da spaziamento + base :

```

struct student
{
    char  name[100];
    int   roll;
    float marks;
};

struct student stu1;
stu1.roll = 12;

```

```

movq $stu1, %rax
movl $12, 100(%rax)

```

Tornando alle struct , dobbiamo fare attenzione !! In quanto possono esistere struct (union) incomplete !! . Ma attenzione , visto che sono incomplete (o meglio non totalmente definite) non si posso istanziare, ma comunque è possibile definirne un puntatore:

struct thing *pt;

Applicando questa dichiarazione, è possibile risolvere i riferimenti circolari tra header differenti per realizzare strutture dati ricorsive , **quindi con questa "tecnica" si nascondono i tipi di dato della struct , quindi si ha una sorta di struct astratta.**

```

person.h:
struct person;

struct person *person_create(struct
person *who, char *name, int age, int
height, int weight);

person.c:
#include "person.h"
struct person {
    char *name;
    int age;
    int height;
    int weight;
};

```

Quindi in base a quanto detto finora, in C (funzioni) il passaggio dei parametri avviene sempre per valore , ricevendo una copia del valore , mentre nel caso di vettori/matrici copia del puntatore , mentre nel caso di struct avviene tramite la copia di tutta la struttura. Andiamo ora a vedere un esempio :

```

struct huge {
    char member[4096];
};

extern void f1(struct huge s);
extern void f2(struct huge *s);

struct huge glbl;

void f(void) {
    f1(glbl);
}

```

```

pushq %rbp
movq %rsp, %rbp
subq $4096, %rsp
movq %rsp, %rdi
movq $glbl, %rsi
movq $512, %rcx
cld
rep movsq
call f1
addq $4096, %rsp
leave
ret

```

Andiamo ad applicare quanto detto finora , o meglio andiamo a vedere il movimento delle stringhe in assembler : per farlo si usano due istruzioni dedicate : **movs e stos** : entrambe utilizzano registri impliciti :

1. RCX: contatore del numero di operazioni da eseguire
2. RSI: indirizzo sorgente
3. RDI: indirizzo destinazione
4. RAX: valore cui impostare la memoria
5. DF (direction flag): identifica operazione da svolgere
 - a. Se 0 si va in avanti
 - b. Se 1 si va indietro

Vediamo un esempio :

movs: move data from string to string stos: store string

```

movq $source, %rsi
movq $destination, %rdi
movq $size/8, %rcx
cld
movsq

```

```

movq $0x0, %rax
movq $destination, %rdi
movq $size/8, %rcx
cld
stosq

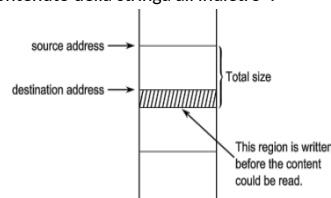
```

Attenzione però a "muovere le stringhe" : se nel caso che la src e la dst sono sovrapposte : si ha inconsistenza, quindi si opta per copiare il contenuto della stringa all'indietro :

```

movb $0x800, %rsi
movb $0x810, %rdi
movb $0x20, %rcx
cld
movsb

```



Analogamente in C:


```
#include <string.h>

void *memcpy(void *restrict dest, const void *restrict src, size_t n);
    Effettua una copia memoria/memoria in avanti

void *memmove(void *dest, const void *src, size_t n);
    Effettua una copia memoria/memoria all'indietro (i buffer possono
    sovrapporsi)

void *memset(void *s, int c, size_t n);
    Imposta un'area di memoria al valore c
```

Vediamo ora come si manifesta la relazione tra vettori e puntatori in C:

```
#include <stdio.h>

int main()
{
    int nums[]={10,12,13,14,20};
    //notare che il '\0' e carattere terminatore
    char name[]={'A','l','e','s','s','a','n','d','r','o','\0'};
    char *name_ptr=name;

    printf("the size of an int: %ld\n",sizeof(int));
    printf("The size of nums (int[]): %ld\n", sizeof(nums));
    printf("The number of ints in nums: %ld\n\n", sizeof(nums) / sizeof(int));
    printf("The size of a char: %ld\n", sizeof(char));
    printf("The size of name (char[]): %ld\n", sizeof(name));
    printf("The number of chars: %ld\n\n", sizeof(name) / sizeof(char));
    printf("The size of name_ptr: %ld\n", sizeof(name_ptr));
    //assegnazione di vettore a puntatore -> autocast
    //vettore quindi decade a puntatore
    //indirizzo primo elemento viene castato ad indirizzo
    //perdo info sul tipo -> name vettore di certo numero di caratteri
    printf("Bugged number of chars: %ld\n", sizeof(name_ptr) / sizeof(char));

    return 0;
}
```

Con output il seguente :

```
the size of an int: 4
The size of nums (int[]): 20
The number of ints in nums: 5

The size of a char: 1
The size of name (char[]): 11
The number of chars: 11

The size of name_ptr: 8
Bugged number of chars: 8
```

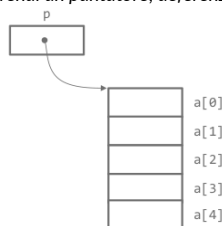
Ricordiamo qui che il "4" sta ad indicare 32 bit!! Invece nell'ultima stampa (bugged) stampa comunque 8 (indirizzo a 64 bit) , ma viene convertito grazie ad autocast -> **si perdono informazioni sul tipo di dato**

Quindi con questa assegnazione (autocast) da vettore a puntatore, come già detto si perdono delle info e queste info sono la taglia del vettore. Quindi stesso comportamento , ma cambia il codice assembler .

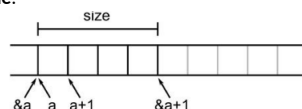
Esempio :

```
int a[4] = {3, 2, 1, 0};
int *p = a;
```

Da notare che qui entrambi generano l'indirizzo dove elemento 2 è memorizzato . Quindi in generale e riassumendo : un puntatore può essere deferenziato utilizzando operatore spiazzamento ([]). Quindi in generale : *Prendi un puntatore, deferenzialo , spiazzati da quel punto come se avessi un vettore :*



Analogamente ai puntatori , andiamo a vedere il duale : **L'indirizzo** di una variabile : oggetto duale del puntatore . **Questo oggetto rappresenta effettivamente la cella di memoria dove si trova quella variabile.**



Vediamo un esempio :

file1.c:

```
#include <stdio.h>

extern int *array;

int main(void)
{
    printf("Third element of the
    array is %d\n", array[2]);
    return 0;
}
```

file2.c:

```
int array[] = {0, 1, 2, 3, 4};
```

- Questi due file compilano in un programma correttamente
- Non generano nemmeno un warning
- Il programma è corretto?

Il programma assolutamente sbagliato !! Core dump creato .

Vediamo altro esempio :

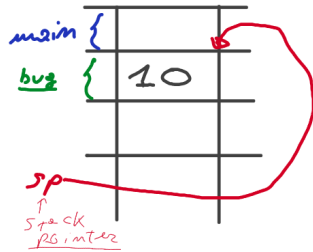
```
#include <stdio.h>

int *buggy_return(void)
{
    int a_variable=10;
    //ritorna indirizzo contenente variabile nello stack
    return &a_variable;
}

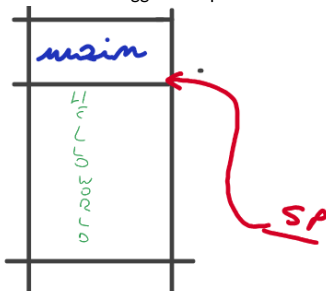
void using_stack(void)
{
    char a_string[]="Hello World!";
    printf("%s\n",a_string);
}

int main(void)
{
    int *dummy=buggy_return();
    //deferenzio il puntatore , accedendo al valore
    printf("%d\n",*dummy);
    using_stack();
    printf("%d\n",*dummy);
    return 0;
}
```

Il programma va in crash : core dump. Invalida la finestra di stack. Quindi nel caso del codice sopra, si hanno le finestre di stack sovrapposte. Vediamolo in dettaglio cosa succede alla finestra di stack : Nel primo disegno si ha la situazione nella funzione BUGGY_RETURN:



Mentre nel prossimo sia ha la situazione dello stack della funzione STACK_PRINTER : da notare che l'area dello stack allocata è maggiore in quanto è diverso il parametro :

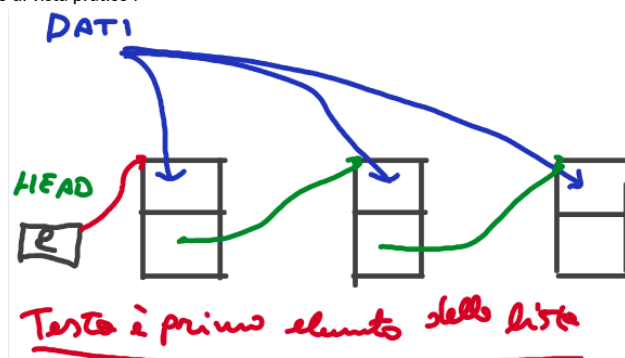


Vediamo ora un altro concetto : **le liste** : codice compatto ed efficiente . Si parla di quelle singolarmente collegata : insieme ordinati di nodi :



```
struct node_t {
    int data;
    node_t * next;
};
```

Concettualmente è corretto , ma praticamente no : sia per il fatto che si punta a null e secondo le frecce (puntatori a strutture) : dovrebbero contenere indirizzo del primo byte del nodo successivo . Quindi dal punto di vista pratico :



Da notare che all'interno di head vi è un indirizzo : è un puntatore ad indirizzo della struct. Vediamo ora un esempio di codice : rimozione di un nodo:

```
struct node_t {
    int data;
    node_t * next;
};

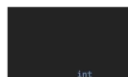
void remove_entry(struct node_t **const head, struct node_t const *const entry)
{
    struct node_t *prev = NULL;
    struct node_t *walk = *head;
    while(walk != entry) {
        prev = walk;
        walk = walk->next;
    }

    if(!prev) {
        *head = entry->next;
    } else {
        prev->next = entry->next;
    }
}
```

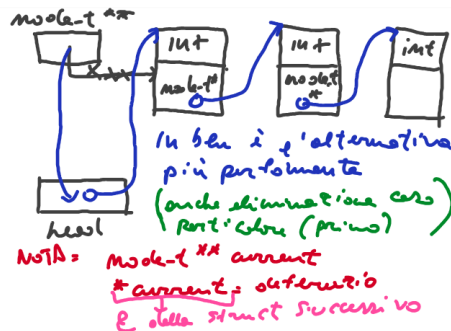
NOTA: null=0=false.

Si evince da qui che walk indica il puntatore che indica il cammino ; -> indica che leggo il contenuto. Inoltre non basta un solo puntatore , in quanto se devo eliminare il primo elemento , come si fa? Vediamo una versione migliorata :

```
void remove_entry(struct node_t **head, struct node_t const *const entry)
{
    while((*head) != entry) {
        head = &(*head)->next;
    }
    *head = entry->next;
}
```



Innanzitutto ingloba il caso particolare , poi evita di avere due puntatori per due elementi adiacenti . **Quindi in questo caso si parla di puntatore a puntatore di struct.** Vediamola ora in modo concreto:



Andiamo a vedere ora la precedenza degli operatori (ordine degli operatori in esecuzione) : vengono scelti dal compilatore :

	Operatori	Associatività
Precedenza (da maggiore a minore)	() [] -> .	da destra a sinistra
	! ~ ++ -- + - * & (type) sizeof	da destra a sinistra
	* / %	da sinistra a destra
	+ -	da sinistra a destra
	<< >>	da sinistra a destra
	< <= > >=	da sinistra a destra
	== !=	da sinistra a destra
	&	da sinistra a destra
	^	da sinistra a destra
		da sinistra a destra
	&&	da sinistra a destra
		da sinistra a destra
	?:	da destra a sinistra
	= += -= *= /= %= &= ^= = <<= >>=	da destra a sinistra
	,	da sinistra a destra

Come per esempio :

Espressione	Equivalente a	Significato
*p++	*(p++)	p viene incrementato dopo il suo utilizzo per accedere alla memoria.
*++p	*(++p)	p viene incrementato prima del suo utilizzo per accedere alla memoria.
++*p	++(*p)	p è utilizzato per l'accesso in memoria, ma il contenuto della locazione viene incrementato prima del suo utilizzo.
(*p)++	(*p)++	p è utilizzato per l'accesso in memoria; il contenuto della locazione viene incrementato dopo il suo utilizzo.

Vediamo ora come creare alias (nomi alternativi a tipi di dato esistenti) alle struct: **il typedef**

```
struct var {
    int data1;
    int data2;
    char data3;
};

struct var a;

typedef struct var newtype;

newtype a;
```

Andiamo ora a vedere un altro tipo di struttura in c: la **union** : valore che può assumere una qualsiasi configurazione in una stessa porzione di memoria. In altre parole un'unione è la sovrapposizione di tipi

di dati differenti nella stessa area di memoria , quindi con una variabile posso rappresentare più tipi di dato , ma questo tipo di dato è uno solo (assomiglia al polimorfismo di java):

```
#include <stdio.h>

typedef enum type{
    INT,
    FLOAT,
    DOUBLE
}type_t;

typedef union operand{
    //nella struttura ci puo essere un solo tipo
    int i;
    float f;
    double d;
}operand_t;

void sum(type_t t, operand_t op1, operand_t op2)
{
    switch(t) {
        case INT:
            printf("%d + %d = %d\n", op1.i, op2.i, op1.i + op2.i);
            break;
        case FLOAT:
            printf("%.02f + %.02f = %.02f\n", op1.f, op2.f, op1.f + op2.f);
            break;
        case DOUBLE:
            printf("%.02lf + %.02lf = %.02lf\n", op1.d, op2.d, op1.d + op2.d);
            break;
        default:
            printf("Unexpected operand types.\n");
    }
}

int main(void)
{
    // Designated union initializers
    operand_t
    op11 = { .i = 3 },
    op12 = { .i = 5 },
    op21 = { .f = 2.18f },
    op22 = { .f = 8.79f },
    op31 = { .d = 1.53 },
    op32 = { .d = 4.71 };
    sum(INT, op11, op12);
    sum(FLOAT, op21, op22);
    sum(DOUBLE, op31, op32);
    printf("sizeof(int) = %zu\n", sizeof(int));
    printf("sizeof(float) = %zu\n", sizeof(float));
    printf("sizeof(double) = %zu\n", sizeof(double));
    printf("sizeof(operand_t) = %zu\n", sizeof(operand_t));

    return 0;
}
```

Con il seguente output :

```
3 + 5 = 8
2.18 + 8.79 = 10.97
1.53 + 4.71 = 6.24

sizeof(int) = 4
sizeof(float) = 4
sizeof(double) = 8
sizeof(operand_t) = 8
```

Notiamo che con queste strutture possiamo fare diversi cast :

```
#include <stdio.h>
#include <stdint.h>

union binary_float_t {
    float real;
    uint32_t integer; // Assumes float is 32 bits wide
};

int main(void)
{
    union binary_float_t f;
    f.real = 3.141592f;
    printf("Hex representation of %f is %08x\n", f.real, f.integer);
    return 0;
}
```

Con output seguente :

```
Hex representation of 3.141592 is 0x40490fd8
```

Sempre usando le struct o meglio le union, come possiamo rappresentare dati minori di 1 byte(8 bit)??

Usando la tecnica del **bit fielding** : rappresento gruppi di n-bit all'interno dei quali ne uso solamente una porzione :

```
union f1t {
    struct ieee754 {
        uint32_t mantissa: 23;
        uint32_t exponent: 8;
        uint32_t sign: 1;
    } raw;
    float f;
};

number.raw.sign = 1;
number.raw.exponent = 120;
number.raw.mantissa = 1685475;

printf("\nConverting %d %s %s to float:\n", number.raw.sign, exponent,
    mantissa);
printf("\t%f\n", number.f);
```

Quindi più in generale , esistono dei meccanismi per forzare un qualunque valore ad un qualunque valore : **Maschera di bit** : o sotto il bit più significativo a 0 oppure a 1 . Nel caso dell' "1" si usa solo in assembler con l'istruzione : **orl \$0x80000000, %eax** , mentre per settare la maschera di bit a "0" si usa **andl \$0x7FFFFFFF, %eax** . Per quanto riguarda invece il procedimento (invero ultimo bit) uso **xorl \$0x80000000, %eax** . Vediamo un esempio :

Supponiamo di avere un numero a 32 bit e di voler estrarre il valore dei 3 bit meno significativi
Si costruisce una maschera di bit del tipo 000...00111, equivalente a 7
Estrazione dei bit: **andl \$7, %eax**
Per verificare se i bit sono a zero: **testl \$7, %eax**

Che cosa fa l'istruzione **testq %rax, %rax**?

Andl ritorna gli ultimi tre bit meno significativi !! . Solo 3 bit perché abbiamo scelto di leggere solo "7" in codifica binaria. Con testl verifico se almeno uno degli n-bit vale 1 , con aggiornamento dello zf (zero-flag). Con l'ultima istruzione calcolo and bit-a-bit !! . **Se 0 and 0 lo zero flag vale 1. Andiamo ora a vedere come risolvere il seguente problema : per accedere ad un bit , devo "percorrere la gerarchia di livelli" , e**

se questa gerarchia è grande, aumenta il tempo: usiamo le union / struct anonime: strutture come se fossero a livello inferiore:

```
#include <stdio.h>

struct scope
{
    // Anonymous structure
    struct
    {
        char alpha;
        int num;
    };
};

int main()
{
    struct scope x;
    x.num = 65;
    x.alpha = 'A';
    printf("x.alpha = %c, x.num = %d\n", x.alpha, x.num);

    return 0;
}
```

Con output il seguente:

```
x.alpha = A, x.num = 65
```

Il programmatore può dire al compilatore come trattare le variabili, così in modo che il compilatore possa effettuare maggiore ottimizzazione:

const (C89): indica che i dati sono in sola lettura.

volatile (C89): indica che un valore può cambiare tra due accessi diversi, anche se apparentemente nulla l'ha cambiato (previene ottimizzazioni del compilatore).

restrict (C99): si utilizza solo con dichiarazioni di puntatori. Indica al compilatore che, per tutta la vita del puntatore, solo questo avrà accesso all'oggetto puntato.

Analogamente a quanto detto per le variabili, ovvero si usano i puntatori per salvarne il valore, esistono anche **puntatori a funzione**: variabili a cui possiamo assegnare il valore dell'indirizzo in memoria delle funzioni:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define ELEMENTS 6

int values[] = { 40, 10, 100, 90, 20, 25 };
// puntatore a funzione
typedef int (*compare_t)(const void *, const void *);
/*
 * retval meaning
 * <0 The element pointed to by p1 goes before the element pointed to by p2
 * 0 The element pointed to by p1 is equivalent to the element pointed to by p2
 * >0 The element pointed to by p1 goes after the element pointed to by p2
 */
int compare(const void *a, const void *b) { return (*(int *)a - *(int *)b); }

void shuffle(int *array, size_t n)
{
    //ordino a casaccio il vettore
    if (n > 1) {
        size_t i;
        for (i = 0; i < n - 1; i++) {
            size_t j = i + rand() / (RAND_MAX / (n - i) + 1);
            int t = array[j];
            array[j] = array[i];
            array[i] = t;
        }
    }
}

void print_array(char *header, int *array, size_t n)
{
    int i;
    printf("%s: ", header);
    for (i = 0; i < n; i++)
    {
        printf("%d ", values[i]);
        puts("");
    }
}

int main(int argc, char *argv[])
{
    compare_t compare_f1 = compare; // Indirizzo della prima istruzione della funzione. Attenzione al tipo
    int (*compare_f2)(const void *, const void *) = compare; // dichiarazione esplicita
    srand(time(0));
    print_array("Original", values, ELEMENTS);
    qsort(values, ELEMENTS, sizeof(int), compare);
    print_array("Sorted with f.name", values, ELEMENTS);
    shuffle(values, ELEMENTS);
    print_array("Shuffled", values, ELEMENTS);
    qsort(values, ELEMENTS, sizeof(int), compare_f1);
    print_array("Sorted with f.ptr 1", values, ELEMENTS);
    shuffle(values, ELEMENTS);
    print_array("Shuffled", values, ELEMENTS);
    qsort(values, ELEMENTS, sizeof(int), compare_f2);
    print_array("Sorted with f.ptr 2", values, ELEMENTS);
    return 0;
}
```

Il cui output è il seguente:

```
Original: 40
10
100
90
20
25
Sorted with f.name: 10
20
25
40
90
100
Shuffled: 90
100
25
40
20
10
Sorted with f.ptr 1: 10
20
25
40
90
100
Shuffled: 100
20
```

```

20
25
90
40
10
Sorted with f_ptr 2: 10
20
25
40
90
100

```

Riprendendo il discorso dei puntatori in C, andiamoli ora a vedere in assembler : il loro uso è molto facile, in quanto il passaggio dei parametri viene effettuato molto velocemente, grazie alle calling conventions. Quindi nel caso di funzione parametrica in assembler basta fare :

```

movq $function, %rax
call *%rax

```

Riepilogando invece per quanto riguarda i qualificatori di tipo ed i puntatori : i primi servono a specificare i tupa di puntatori utilizzati e l'area di memoria puntata. Mentre nel secondo servono a specificare il puntatore stesso ed il tipo di dato puntato . Inoltre i primi (qualificatori di tipo) servono anche per interpretare la scrittura del codice ed i tipi di dato , visto che rendono molto versatile il linguaggio . Vediamo ora degli esempi :

```

int *ptr: un puntatore a intero
int const *ptr: un puntatore a intero costante
const int *ptr: un puntatore a intero costante
int * const ptr: un puntatore costante a intero
int const * const ptr: un puntatore costante a intero costante
const int * const ptr: un puntatore costante a intero costante
int **ptr: un puntatore a un puntatore a intero
int ** const ptr: un puntatore costante a un puntatore a intero
int * const * ptr: un puntatore a un puntatore costante a intero
int const **ptr: un puntatore a un puntatore a un intero costante
int * const * const ptr: un puntatore costante a un puntatore
costante a intero

```

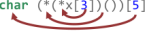
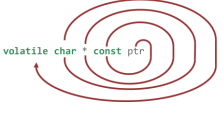
Ma da notare che in generale è un bel rompicapo capire cosa si voglia dire attraverso queste istruzioni :

```

char **argv: un puntatore a puntatore a carattere
const char * const * const ptr: un puntatore costante a un
puntatore costante a un carattere costante
volatile char * const ptr: un puntatore costante a un carattere
volatile
int *ptr[13]: un vettore di 13 puntatori a intero
int (*ptr)[13]: un puntatore a un vettore di 13 interi
void *ptr(): una funzione che restituisce un puntatore generico
void (*ptr)(): un puntatore a una funzione senza valore di ritorno
char (*x())[](): una funzione che restituisce un puntatore a un
vettore di puntatori a funzione che restituiscono un carattere
char ((*x[3])())[5]: un vettore di tre puntatori a funzione che
restituiscono un puntatore ad un vettore di cinque caratteri

```

Quindi , vista la complessità della traduzione si usa la famosa **regola della spirale** : si parte dal nome e si procede in senso orario :

 str è un vettore di 10 puntatori a carattere	 fp è un puntatore a una funzione che accetta un intero e un puntatore a float che restituisce un puntatore a carattere
 signal è una funzione che accetta un intero e un puntatore a una funzione che accetta un intero e non restituisce nulla che restituisce un puntatore a una funzione che accetta un intero e non restituisce nulla	 x è un vettore di tre puntatori a funzione che restituiscono un puntatore ad un vettore di cinque caratteri
 ptr è un puntatore costante a un carattere volatile	

Il linguaggio c , consente inoltre di dichiarare **funzioni variadiche** : ovvero funzioni che accetta un numero arbitrario di parametri , mentre la stampa lo è intrinsecamente .

Funzioni variadiche

```

#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>

int somma(int n, ...)
{
    int nSum = 0;
    va_list int_ptr;
    va_start(int_ptr, n);
    for (int i = 0; i < n; i++)
        nSum += va_arg(int_ptr, int);
    va_end(int_ptr);
    return nSum;
}

int main(int argc, char **argv) {
    printf("10 + 20 = %d\n", somma(2, 10, 20));
    printf("10 + 20 + 30 = %d\n", somma(3, 10, 20, 30));
    printf("10 + 20 + 30 + 40 = %d\n", somma(4, 10, 20, 30, 40));
    return EXIT_SUCCESS;
}

```

Da notare l'header <stdarg.h>: permette di accedere ad un nuovo tipo che si chiama va_list (lista argomenti variadici) , la quale permette al programma di tenere traccia tra un numero di argomenti , di

sapere qual è l'ultimo usato. Questa variabile viene inizializzata con il numero dei parametri. Andiamo a vederlo in assembler : i primi 6 parametri passati per registro , mentre gli altri per comunicazione con stack : quindi vista la complessità nell'accesso allo stack , va_start effettua una copia di tutti quanti i parametri dai registri allo stack : da notare che se i parametri maggiori di 6 , quelli dopo il sesto sono ad indirizzo più basso di quello del base pointer (ricordiamoci che gli indirizzi nello stack partono da valori alti ed arrivano a valori bassi). Al termine della funzione , i parametri vengono eliminati dallo stack(per i primi 6 ci pensa va_start), mentre per quelli eccedenti il programmatore. Andiamo a vedere un esempio riassuntivo : in assembly abbiamo la funzione **movs**: copia di buffer di memoria in un altro buffer; mentre in c si ha la **memcpy(char *from, char *to, int count)**:

```
int memcpy(char *from, char *to, int count)
{
    int i = 0;
    for (i = 0; i < count; i++) {
        to[i] = from[i];
    }
    return i;
}
```

Nota : questa rappresentazione fa schifo !! Andiamo a vedere come renderlo più performante :

```
int duff_device(char *from, char *to, int count)
{
    int n = (count + 7) / 8;
    switch (count % 8) {
        case 0:
            do {
                *to++ = *from++;
            } while (--n > 0);
        case 7:
            *to++ = *from++;
        case 6:
            *to++ = *from++;
        case 5:
            *to++ = *from++;
        case 4:
            *to++ = *from++;
        case 3:
            *to++ = *from++;
        case 2:
            *to++ = *from++;
        case 1:
            *to++ = *from++;
    }
    return count;
}
```

Andiamo a vedere ora la **memoria dinamica** : area di memoria dalla quale possiamo recuperare dei buffer nel momento in cui ci servono. Questa area di memoria prende il nome di HEAP : ovvero area di memoria a noi sconosciuta in quanto viene allocata a tempo di compilazione, quindi fornisce a tempo di esecuzione tali blocchi. Quindi il programmatore deve "richiedere" buffer di memoria e rilasciarli quando non ci servono più. In C si fa ciò usando due funzioni di libreria :

void *malloc(size_t size): alloca una quantità di memoria di dimensione size byte e restituisce un puntatore alla memoria allocata, o NULL in caso di errore. La memoria non è inizializzata. Se size è zero, malloc() restituisce NULL o un puntatore che può essere successivamente passato a free().
void free(void *ptr): libera l'area di memoria puntata da ptr, che deve essere stato precedentemente restituito da malloc() (o varianti).

Vediamo un esempio :

```
int *ptr = malloc(10 * sizeof(int));
if (!ptr) {
    /* Manage the error here */
} else {
    /* Allocation successful. Do whatever you want! */
    free(ptr); /* When memory is not needed anymore, you free it. */
    ptr = NULL; /* Set the pointer to NULL, to avoid a "dangling" pointer */
}
```

Nota la malloc non inizializza l'area di memoria , quindi se ne deve occupare il programmatore. Questa possibilità di allocare aree di memoria dinamicamente permette di avere gli **array flessibili** ovvero un vettore di cui non si specifica la dimensione (concettualmente di dimensione 0) :

```
struct list {
    struct list *next;
    size_t size;
    unsigned char payload[];
}
```

Quindi per allocare una struttura del genere :

```
malloc(sizeof(struct list) + size)
```

Vediamo ora una combinazione delle precedenti due :

void *calloc(size_t nmemb, size_t size): restituisce un'area di memoria tale da contenere un vettore di nmemb elementi, ciascuno di dimensione size. La memoria è inizializzata a zero. Viene verificato se nmemb * size provoca overflow, caso in cui viene generato un errore.
void *realloc(void *ptr, size_t size): viene "modificata" la quantità di memoria puntata da ptr, effettuando una nuova allocazione e spostando size byte da ptr al nuovo buffer allocato. Il buffer puntato da ptr viene liberato con free(ptr). Il nuovo buffer viene restituito dalla funzione.

Facciamo attenzione al **memory leak** : non rilascio la memoria , aumentando così la possibilità di undefined behaviour:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

bool leak(void)
{
    char *s = malloc(4096);
    if (s == NULL) {
        return false;
    } else {
        s[0] = 'A';
        return true;
    }
}

int main(void)
{
    while(leak());
    return 0;
}
```

Attenzione quando passo il puntatore a free : il puntatore contiene indirizzo . **Quindi attenzione ai possibili bugs** : nonostante l'area di memoria logicamente invalidata ,se non uso il free sul puntatore, quella stessa area di memoria potrebbe essere riusata. Andiamo ora a vedere tutti i possibili casi si bugs : **undefined behaviour** : lo evito annullando il puntatore:

```
int *ptr = malloc(sizeof(int));
free(ptr);
*ptr = 0; /* undefined behavior */
printf("%p", ptr); /* undefined behavior */
```

Oppure lo evito usando **lo scope delle variabili** : alloca una nuova variabile in una diversa finestra di stack (diversa locazione di memoria), quindi non sovrascrivo il valore del puntatore!!

Ma attenzione al tipo di puntatore restituito !! (**freeing wrong pointers**): puntatore è di tipo sbagliato (puntatore restituito da malloc -> metadati corrotti (bitmap).

```
char *msg = "Una stringa globale";
int tbl[100];
free(msg); /* undefined behavior */
free(tbl); /* undefined behavior */
```

Vediamo ora la **corruzione da doppio free**(double free corruption) : dato che il free libera solo un buffer di memoria alla volta , se lo faccio due volte sullo stesso puntatore si ha un undefined behaviour :

```
int *ptr = malloc(sizeof(int));
free(ptr);
free(ptr);
```

Andiamo ora a vedere come implementare un vettore dinamico , arrivando cosi' ad esempio di matrice dinamica : andiamo a vederlo in dettaglio : il file vector.c implementa il nostro array dinamico, mentre il main lo esegue ed infine il file vector.h fa da ponte tra tutti questi file .

Andiamo a vedere il codice :

<pre>#pragma once #include <stdlib.h> #include <stdbool.h> //pragma once compilato solo una volta struct array; //dichiarazione del tipo di dato incompleto extern void *init_vector(size_t nmemb, size_t size); //allocazione del vettore dinamico extern bool insert_at(struct array *arr, size_t pos, void *el); //non ho contesto dei dati extern bool push_back(struct array *arr, void *el); //insert in coda extern bool read_from(struct array *arr, size_t pos, void *buffer); //estrae elemento e lo copia nel buffer extern bool extract_from(struct array *arr, size_t pos, void *buffer); //lida a sopra ed in più lo rimuove extern void free_vector(struct array *arr); //rilascio la memoria dinamica allocata precedentemente</pre>	File vector.h
<pre>#include <stdio.h> #include <stdlib.h> #include <errno.h> #include <string.h> #include <stdbool.h> #include <assert.h> #include "vector.h" //header incluso per implementazione compatibile //specifico la nostra struttura "astratta" struct array { size_t count; // currently used slots size_t slots; // total slots size_t size; // size of a slot void *arr; // the actual array };</pre>	Vector.c headers
<pre>void *init_vector(size_t nmemb, size_t size) { //simile alla calloc //gestisco dati di qualsiasi tipo struct array *ret; void *p; // Check for overflow // (size_t)-1 -> max nr rappresentabile @ 64 bit if (size && nmemb > (size_t)-1/size) { //size && nmemb == size*nmemb //size && nmemb calcola anche il caso se size_t=0!! //0 && qualcosa -> 0 errno = ENOMEM; //comunicazione di errore return 0; // 0 è autocastato da null } p = malloc(size * nmemb); //non inizializza area di memoria if(!p) return p; // p->null malloc return errno memset(p, 0, (size*nmemb)); // azzero il buffer puntato da malloc //in asm viene rappresentato con lo stos ret = malloc(sizeof(struct array)); // allocazione vera e propria.conosco come è fatto if(!ret) { free(p); // rilascio il buffer, altrimenti memory leak return ret; } //inizializzo la struct ret->count = 0; ret->slots = nmemb; ret->size = size; ret->arr = p; return ret; }</pre>	Inizializzazio ne del vettore

<pre><code>bool insert_at(struct array *arr, size_t pos, void *el) { //inserisco elemento o in coda o in mezzo char *from, *to; size_t size; //posizione errata if(pos > arr->count) return false; //elemento oltre ultima cella if(arr->count + 1 > arr->slots) //raddoppio la grandezza del vettore //per evitare che vada oltre dimensione if(!resize(arr, arr->slots * 2)) return false; //inserimento a metà //indirizzo partenza del vettore from = (char *)arr->arr + pos * arr->size; //sposta elementi se non in coda if(pos + 1 < arr->slots) { to = from + arr->size; //count-pos=nr elementi che devo spostare size = (arr->count - pos) * arr->size; //qui src e dest si possono sovrapporre memmove(to, from, size); //in asm movs a seconda del df =0-> avanti , altrimenti indietro //std per settare il bit ad uno nel df (registro invisibile) } //copia elemento da from , il valore el allo slot size memcpy(from, el, arr->size); arr->count++; //assert per vedere se non scrivo fuori dal vettore assert(arr->count <= arr->slots); return true; }</code></pre>	Insert in una posizione passata
<pre><code>bool extract_from(struct array *arr, size_t pos, void *buffer) { //estrazione di elemento //riallocazione della struttura char *from, *to; size_t size; if(!read_from(arr, pos, buffer)) return false; if(pos + 1 < arr->slots) { from = (char *)arr->arr + (pos + 1) * arr->size; to = from - arr->size; size = (arr->count - pos) * arr->size; memmove(to, from, size); } arr->count--; if(arr->count < arr->slots / 2) resize(arr, arr->slots / 2); return true; }</code></pre>	Estrae elemento e "aggiusta"
<pre><code>//collegamento solamente interno //inline aumento la leggibilità. Non si ha call a questa funzione //il codice di questa funzione viene messa a posto della call static inline bool resize(struct array *arr, size_t new_size) { assert(arr->count < new_size); //alloca nuovo buffer, copia il contenuto ed aggiorna l'indirizzo //chiama internamente la malloc arr->arr = realloc(arr->arr, new_size * arr->size); if(!arr->arr) return false; arr->slots = new_size; return true; }</code></pre>	Funzione che "mette bene"
<pre><code>void free_vector(struct array *arr) { free(arr->arr); free(arr); } bool push_back(struct array *arr, void *el) { return insert_at(arr, arr->count, el); }</code></pre>	Libera memoria Insert in posizione data

Vediamo ora il main :

<pre>static void init_guy(person_t *guy, char *name, char *surname, unsigned int age, bool employed, bool married, bool has_children, bool student) { strncpy(guy->name, name, 64); strncpy(guy->surname, surname, 64); guy->age = age; guy->employed = employed; guy->married = married; guy->has_children = has_children; guy->student = student; } static void print_guy(person_t *guy) { assert(guy != NULL); printf("%s %s\n", guy->name, guy->surname); printf("age: %u\n", guy->age); printf("employed: %s\n", guy->employed ? "yes" : "no"); printf("married: %s\n", guy->married ? "yes" : "no"); printf("has children: %s\n", guy->has_children ? "yes" : "no"); printf("student: %s\n", guy->student ? "yes" : "no"); }</pre>	Inizializzazione stampa
<pre>int main(void) { //inizializzo la stessa variabile molte volte int i; person_t guy; people = init_vector(2, sizeof(person_t)); init_guy(&guy, "Mario", "Rossi", 48, false, true, true, false); push_back(people, &guy); init_guy(&guy, "Luigi", "Esposito", 12, false, false, false, true); push_back(people, &guy); init_guy(&guy, "Maria", "Bianchi", 34, true, true, true, false); insert_at(people, 0, &guy); init_guy(&guy, "Giuseppina", "Romano", 27, false, true, true, false); insert_at(people, 1, &guy); init_guy(&guy, "Antonio", "Ricci", 76, true, true, false, false); push_back(people, &guy); for(i = 0; i < 5; i++) { if(!extract_from(people, 0, &guy)) { printf("Error retrieving person\n"); continue; } print_guy(&guy); } printf("Have records: %s\n", read_from(people, 0, &guy) ? "yes" : "no"); free_vector(people); return 0; }</pre>	Main ed esecuzione con vari esempi
<pre>Maria Bianchi age: 34 employed: yes married: yes has children: yes student: no Giuseppina Romano age: 27 employed: no married: yes has children: yes student: no Mario Rossi age: 48 employed: no married: yes has children: yes student: no Luigi Esposito age: 12 employed: no married: no has children: no student: yes Antonio Ricci age: 76 employed: yes married: yes has children: no student: no Have records: no</pre>	Stampa a schermo

Andiamo ora a vedere come fare i/o con/dispositivi . In C si usa un costrutto standard , ovvero la funzione **scanf()**: ovvero una funzione variadica che accetta un numero di parametri , ma attenzione al numero degli specificatori. Quindi questa funzione permette di leggere dei caratteri dal canale di i/o : li converte secondo lo specificatore passato ed infine memorizza i valori dentro questi specificatori. **Attenzione però al comportamento indefinito !!**. Per comportamento indefinito si intende se il numero di argomenti è maggiore di quello degli specificatori, oppure il contrario . **Nota: per il printf/scanf la stringa termina con '\0' , mentre per stringa normale (%s) termina con '\0'**. Vediamo degli esempi : Ricordiamo gli header e che è stato realizzato dentro uno switch:

<pre>case 1: { //se con input numerico funziona //sia positivo che negativi //se con lettere return 0: // non processa input // -> stack vuoto //undefined behaviour int a; printf("enter a number: "); scanf("%d", &a); printf("You entered %d.\n", a); break; }</pre>	<pre>case 2: { //con numeri funziona //con caratteri entra //in ciclo infinito!! int a; printf("enter a number: "); //si usa solo per scan formattate (da file) while (scanf("%d", &a) != 1) { // input was not a number, ask again: printf("enter a number: "); } printf("You entered %d.\n", a); break; }</pre>
--	---

<pre> case 3: { //scrive dati su stdin-> //potrebbe portare ad inconsistenza int a; printf("enter a number: "); while (scanf("%d", &a) != 1) { // input was not a number, ask again: fflush(stdin); // <- never ever do that! printf("enter a number: "); } printf("You entered %d.\n", a); break; } </pre>	<pre> case 4: { //se nome < 12 char ok //se nome con spazio solo il primo //se nome con separatore != " " tutto char name[12]; printf("What's your name? "); scanf("%s", name); printf("Hello %s!\n", name); break; } </pre>
<pre> case 5: { //cosi funziona anche se spazi come separatore char name[40]; printf("What's your name? "); scanf("%39[^\n]", name); // note the space here, matching any whitespace printf("Hello %s!\n", name); break; } </pre>	<pre> case 6: { //soffre l'enter char name[40]; printf("What's your name? "); if (fgets(name, 40, stdin)) { printf("Hello %s!\n", name); } break; } </pre>
<pre> case 7: { //trova indice che matcha con '\n' char name[40]; printf("What's your name? "); if (fgets(name, 40, stdin)) { name[strcspn(name, "\n")] = 0; printf("Hello %s!\n", name); } break; } </pre>	<pre> case 8: { //qui se input e carattere/stringa entra //in ciclo finche non trova un numero //funziona anche se '\n' oppure '\0' int a; char buf[1024]; // use 1024 just to be sure do { printf("enter a number: "); if (fgets(buf, 1024, stdin)) { // reading input failed, give up: return 1; } // have some input, convert it to integer: a = atoi(buf); } while (a == 0); // repeat until we got a valid number printf("You entered %d.\n", a); break; } </pre>
<pre> case 9: { //questa versione migliorata bypassa il bug del '\n' e del '\0' long a; char buf[1024]; // use 1024 just to be sure int success; // flag for successful conversion do { printf("enter a number: "); if (fgets(buf, 1024, stdin)) { // reading input failed: return 1; } // have some input, convert it to integer: char *endptr; errno = 0; // reset error number a = strtol(buf, &endptr, 10); if (errno == ERANGE) { printf("Sorry, this number is too small or too large.\n"); success = 0; } else if (endptr == buf) { // no character was read success = 0; } else if (*endptr && *endptr != '\n') { // *endptr is neither end of string nor newline // so we didn't convert the *whole* input success = 0; } else { success = 1; } } while (!success); // repeat until we got a valid number printf("You entered %ld.\n", a); break; } </pre>	<pre> Default: { printf("\n leggi cosa vuoi fare ?? \n"); printf("\n"); printf("\t1) leggi un numero \n"); printf("\t2) leggi un numero con ciclo\n"); printf("\t3) leggi un numero e mandalo su stdin \n"); printf("\t4) leggi una stringa \n"); printf("\t5) leggi una stringa formattata \n"); printf("\t6) leggi qualcosa \n"); printf("\t7) leggi ed identifica terminatori \n"); printf("\t8) leggi un numero senza gestione \n"); printf("\t9) leggi un numero con gestione \n"); printf("\n"); } </pre>

Ricordiamo che `errno` (error number) mantiene un riferimento ad ultimo errore verificato, mentre `strtol` (str to long) trasforma da byte alfanumerici a numerici. In generale come parametri dello `strtol` sono : un buffer dove scrivere, l'indirizzo della stringa che deve convertire, mentre l'ultimo rappresenta la base nella quale fare la conversione. In alternativa allo `scanf` vi sarebbe altra funzione : **la gets()**, **ma non si usa in quanto è piena di bug**. Andiamo ora a vedere invece il **preprocessore C**: ovvero un preprocessore di macro per i linguaggi di programmazione : più in generale si intendono le direttive di compilazione , le quali supportano anche la gestione del codice multi piattaforma. In dettaglio :

1. Inclusione di file
 - a. Permette di includere file esterni a quel sorgente : se il file è tra "<" e ">" lo cerca nei percorsi standard del compilatore, altrimenti se incluso tra " " lo cerca nella stessa cartella del sorgente corrente
2. Compilazione condizionale
 - a. Servono a dire al compilatore di compilare secondo una specifica piuttosto che un'altra :

```

#ifdef __unix__
#include <unistd.h>
#elif defined _WIN32
#include <windows.h>
#endif

```

- ii. Permette di includere header `unistd.h` se SO è linux, altrimenti `windows.h` se è windows

3. Guardie di inclusione
 - a. Consentono di scegliere se e quando includere i file, quindi di definire simboli e tipi

```

#ifndef _FILENAME_H
#define _FILENAME_H
...
#endif

```
 - b.  `#pragma once`

- c. Con la `#pragma once` si intende che il file sarà **compilato una ed una sola volta**.

4. Macro

- a. Servono a definire delle "costanti" : o variabili oppure funzioni. Una volta definite ogni riferimento a quella variabile eseguirà quel codice , oppure avrà quel valore.

```
#include<stdlib.h>
#include<stdio.h>

#define MAX(a,b) ((a) > (b) ? (a) : (b))
#define MIN(a,b) ((a) < (b) ? (a) : (b))

//per escludere una macro si usa #undef<macro>

int main(void)
{
    int x=20;
    int y=10;
    int max=MAX(x++,y);
    int min=MIN(x,y++);
    printf( "max tra x++: %d e y:%d  %d\n",x,y,max);
    printf( "min tra x:%d e y++:%d  %d\n",x,y,min);
    return 0;
}
```

- c. Con output il seguente:

- d.

```
max tra x++: 22 e y:12  21
min tra x:22 e y++:12  11
```